МОСКОВСКИЙ АВИАЦИОННЫЙ ИНСТИТУТ
(НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ)

Институт №8 «Компьютерные науки и прикладная математика»
Кафедра 806 «Вычислительная математика и программирование»

Лабораторная работа №3
по курсу “Компьютерная графика”

Текстурирование

Выполнила: Я. Р. Тарабрина
Группа: М8О-309БВ-24
Преподаватель: В. Д. Бахарев

Москва, 2026

Условие

Цель работы: загрузить изображения из файловой системы, создать из них текстуры и, используя текстурные координаты в вершинах и сэмплеры, наложить их на объекты сцены.

Вариант 10: правильный додекаэдр.

Программа написана на C++20 с использованием Vulkan, GLFW и Dear ImGui на основе стартового проекта и кода второй лабораторной работы. Для декодирования изображений используется библиотека stb_image. Додекаэдры и пол из прошлой работы освещаются по модели Блинна-Фонга и покрыты текстурами, которые выбираются в интерфейсе.

Базовое задание: одна текстура вместо сплошного цвета; текстурные координаты у вершин, созданные процедурно; свойства моделей не в push-константах, а в динамическом uniform buffer; вершинные и индексные данные в памяти GPU; один набор дескрипторов с динамическим uniform buffer и парой «изображение–сэмплер».

Выполнены все дополнительные задания:

1. семь материалов (текстур) на выбор для каждого объекта и для пола, у каждого материала свой набор дескрипторов;
2. нетривиальное сэмплирование у одного объекта: несколько образцов одной текстуры с разными координатами, зависящими от времени, и модуляция цвета; включается флагом в push-константах;
3. карты блика (specular) и свечения (emissive) рядом с альбедо; если карты нет, вместо неё используется текстура из одного чёрного пикселя.

Кроме этого, на сцене есть пол с повторяющейся текстурой и три объекта, движущиеся по траекториям из первой работы.

Исходный код: https://github.com/batsuperflash/cg-labs, папка `lab03`.

Метод решения

Текстурные координаты

Каждая вершина получила третий атрибут — текстурные координаты (u, v) (формат `R32G32_SFLOAT`, `location = 2`, 8 байт, шаг

вершины стал 32 байта). Они создаются процедурно вместе с позициями и нормалями.

Додекаэдр. Грань — правильный пятиугольник, у каждой грани свои 5 вершин. Для грани уже есть центр $\boldsymbol{c}$, базис $(\boldsymbol{u}, \boldsymbol{w})$ в её плоскости с $\boldsymbol{w} = \boldsymbol{n} \times \boldsymbol{u}$ и радиус описанной окружности R. Вектор от центра к вершине $\boldsymbol{d} = \boldsymbol{p} - \boldsymbol{c}$ переводится в координаты текстуры так, чтобы пятиугольник оказался вписан в круг радиуса 0,5 с центром $(0{,}5;\ 0{,}5)$:

$$u = 0{,}5 + 0{,}5\,\frac{\langle \boldsymbol{d}, \boldsymbol{u} \rangle}{R}, \qquad v = 0{,}5 - 0{,}5\,\frac{\langle \boldsymbol{d}, \boldsymbol{w} \rangle}{R}.$$

Тройка $(\boldsymbol{u}, \boldsymbol{w}, \boldsymbol{n})$ правая, поэтому при взгляде на грань снаружи $\boldsymbol{u}$ направлен вправо, а $\boldsymbol{w}$ вверх. В изображении строка с $v = 0$ — верхняя, то есть v растёт вниз. Знак минус перед вторым слагаемым как раз это учитывает: без него текстура на каждой грани была бы отражена по вертикали. Так как все грани одинаковы, на каждой видна одна и та же картинка с центром в центре грани; направление $\boldsymbol{u}$ у разных граней разное, поэтому картинка на соседних гранях повёрнута по-разному.

Пол — квадрат $[-1{,}1]^2$ в плоскости XZ, координаты $(u, v) = ((x + 1)/2,\ (z + 1)/2)$: вся текстура один раз. Чтобы текстура на большом полу повторялась, вершинный шейдер умножает координаты на число повторений из данных модели (`texture_params.xy`), а сэмплер повторяет изображение за пределами [0,1] (режим адресации `REPEAT`). **Куб** значков света получает на каждой грани всю текстуру целиком. У конуса из линий координаты нулевые.

Автоматические тесты проверяют, что у каждой грани додекаэдра все пять вершин лежат на окружности радиуса 0,5, пять сторон пятиугольника в плоскости текстуры равны, центр грани переходит в центр текстуры, а текстура не зеркальна: площадь каждого треугольника в координатах (u, v) имеет знак, противоположный обходу треугольника в пространстве. Та же проверка есть у куба.

Загрузка изображений

Изображение читает функция `image::load`: библиотека stb_image декодирует файл (PNG, JPEG, BMP, TGA и другие форматы) и всегда возвращает четыре канала RGBA8, поэтому серые и RGB-картинки приводятся к одному виду. Строки идут сверху вниз без выравнивания,

как этого ждёт копирование в `VkImage`. Одно изображение из одного пикселя (`image::solid`) создаётся в коде: белое — альбедо встроенного материала «plain», чёрное — вместо отсутствующих карт блика и свечения.

Текстуры лежат в папке `textures` и сгенерированы скриптом `tools/generate_textures.py`: кирпич, лава, мрамор, плитка, дерево и тестовая сетка с буквой «F» (по ней видно ориентацию), размер 512 × 512. Шум Перлина периодичен по обеим осям, поэтому текстуры бесшовные и пол выглядит без швов. Карта свечения лавы построена из шума Перлина: яркими остаются только линии, где значение шума близко к нулю.

Текстура в памяти GPU

Функция `graphics::createTexture` создаёт текстуру в четыре шага.

1. **Staging buffer.** Пиксели копируются в буфер в памяти, доступной процессору (`TRANSFER_SRC`). Сама текстура создаётся как `VkImage` с оптимальным тайлингом в памяти устройства, процессор в неё писать не может.
2. **Копирование.** Одноразовый буфер команд отправляется на очередь, после чего программа ждёт завершения. В нём записаны барьеры, смена layout и команды копирования.
3. **Мип-цепочка.** Уровень i получается из уровня $i-1$ командой `vkCmdBlitImage` с линейной фильтрацией и вдвое меньшим размером. У текстуры 512 × 512 получается 10 уровней, число уровней $\lfloor \log_2 \max(w,h) \rfloor + 1$ считается функцией `mipLevelCount` и проверяется тестом. Перед построением проверяется, что формат поддерживает блит с линейной фильтрацией, иначе остаётся один уровень.
4. **Вид и сэмплер.** Для `VkImage` создаётся `VkImageView`, а сэмплер один на все текстуры: линейная фильтрация при увеличении и уменьшении, линейное смешение мип-уровней (`maxLod` не ограничен), адресация `REPEAT`.

Layout каждого уровня меняется барьерами так, чтобы запись одной операции была видна следующей:

- все уровни перед копированием: `UNDEFINED` → `TRANSFER_DST_OPTIMAL`;
- уровень $i-1$ перед блитом, когда он становится источником: `TRANSFER_DST_OPTIMAL` → `TRANSFER_SRC_OPTIMAL`;
- уровень $i-1$ после блита, когда он готов для шейдера: `TRANSFER_SRC_OPTIMAL` → `SHADER_READ_ONLY_OPTIMAL`;
- последний уровень в конце: `TRANSFER_DST_OPTIMAL` → `SHADER_READ_ONLY_OPTIMAL`.

Формат и цвет. Картинки на диске хранят цвет в sRGB, а освещение считается в линейном пространстве. Альбедо и карта свечения создаются с форматом `R8G8B8A8_SRGB`: при чтении шейдером значения переводятся в линейные автоматически, а мип-уровни при блите усредняются тоже в линейных значениях. Карта блика — это данные, а не цвет, поэтому у неё формат `R8G8B8A8_UNORM`, байты читаются как есть.

Вершины и индексы в памяти GPU

В прошлых работах вершинные и индексные буферы лежали в памяти, доступной процессору. Теперь они создаются функцией `createDeviceBuffer` по тому же приёму, что и текстуры: данные пишутся в staging buffer, затем одноразовый буфер команд выполняет `vkCmdCopyBuffer` в буфер, созданный с `VMA_MEMORY_USAGE_AUTO_PREFER_DEVICE` и без доступа процессора. Staging buffer после копирования удаляется. На этом компьютере (Apple M3) память общая, но Vulkan всё равно показывает тип памяти только с флагом `DEVICE_LOCAL` без доступа процессора, и код работает так же, как на дискретной видеокарте. Буферы, которые обновляются каждый кадр (uniform buffers сцены и моделей, storage buffers источников), остались в памяти процессора: копировать их через staging buffer каждый кадр было бы дороже, чем записывать напрямую.

Динамический uniform buffer

Раньше у каждого рисования был свой uniform buffer и свой набор дескрипторов (57 штук). Теперь данные всех моделей лежат в одном буфере подряд:

$$\text{offset}_k = k \cdot s, \qquad s = \left\lceil \frac{176}{a} \right\rceil a,$$

где 176 — размер `ModelUniforms`, a — `minUniformBufferOffsetAlignment` устройства, а k — номер рисования в кадре. На этом устройстве $a = 16$, поэтому $s = 176$, а на видеокартах с выравниванием 256 шаг был бы 256; шаг считается во время работы, а не зашит в код. Размер буфера — $57 \cdot s$ байт: 8 объектов, пол, значки 16 точечных и 16 прожекторных источников и 16 конусов.

В наборе дескрипторов слот имеет тип `UNIFORM_BUFFER_DYNAMIC`, а размер окна чтения (`range`) равен одному `ModelUniforms`. Нужное смещение передаётся последним аргументом команды привязки наборов:

```
vkCmdBindDescriptorSets(cmd, GRAPHICS, layout, 1, 2, sets, 1, &offset).
```

Набор модели один на все рисования, меняется только `offset`. Буфер заполняется в `render()` после ожидания предыдущего кадра, поэтому его можно перезаписывать. Структура `ModelUniforms` в std140 содержит 176 байт: `mat4 model`, `mat4 normal_matrix`, `vec4 albedo` (оттенок), `vec4 specular` (цвет блика и коэффициент), `vec4 texture_params` (повторение текстуры по u и v и множитель свечения). Размер и смещения проверяются через `static_assert`.

Наборы дескрипторов и материалы

Данные разделены на три набора по частоте изменения:

- **Set 0 — сцена**, один раз за кадр: uniform buffer `SceneUniforms` (камера, рассеянный и направленный свет) и два storage buffers с точечными и прожекторными источниками. Этот набор не изменился.
- **Set 1 — модели**, один набор на всю программу: динамический uniform buffer со всеми `ModelUniforms`.
- **Set 2 — материал**, свой набор у каждого материала: три `COMBINED_IMAGE_SAMPLER`, слот 0 — альбедо, слот 1 — карта блика, слот 2 — карта свечения.

В базовой версии альбедо лежало в том же наборе, что и динамический uniform buffer, и набор был один. Для дополнительного задания пара «изображение–сэмплер» перенесена в отдельный набор материала, а наборы 1 и 2 привязываются одной командой с одним

динамическим смещением (оно относится к единственному динамическому слоту).

Материал — это набор файлов с общим именем в папке `textures`: `<имя>_albedo.png` создаёт материал, `<имя>_specular.png` и `<имя>_emissive.png` подхватываются, если они есть. Поиск не зависит от порядка файлов и от расширения, материалы упорядочены по имени (это проверяют тесты). Встроенный материал «plain» (белое альбедо) всегда стоит первым, всего получается 7 материалов. Для каждого создаётся набор дескрипторов: если карты блика или свечения нет, в слот записывается общая чёрная текстура 1 × 1. Пул дескрипторов рассчитан на 34 набора (сцена, модели и до 32 материалов) и 96 пар «изображение–сэмплер».

В интерфейсе у каждого объекта и у пола есть выпадающий список текстур. У `Material` в сцене лежит только номер материала, а сами текстуры и наборы дескрипторов хранятся в приложении, поэтому логика сцены по-прежнему не зависит от Vulkan. Новые объекты получают материалы по очереди.

Нетривиальное сэмплирование

У одного объекта (в начальной сцене у мраморного) цвет текстуры читается не один раз, а три, с разными координатами, зависящими от времени t (в секундах с запуска). Пусть $T(\cdot)$ — чтение текстуры по координатам:

$$\boldsymbol{c}_1 = T(\boldsymbol{x} + (0{,}08;\ 0{,}05)\,t), \qquad \boldsymbol{c}_2 = T(R(0{,}35\,t)\,(\boldsymbol{x} - 1/2)\,s(t) + 1/2), \quad s(t) = 1{,}6 + 0{,}5\sin 0{,}7t,$$

$$\boldsymbol{c}_3 = T(\boldsymbol{x} + 0{,}025\,(\sin(20v + 2t),\ \cos(20u + 2t))).$$

Первый образец — текстура, плывущая по диагонали. Второй — текстура, вращающаяся вокруг центра грани с пульсирующим масштабом ($R(\alpha)$ — матрица поворота). Третий — искажение координат бегущей волной. Образцы смешиваются, а результат умножается на цвет, оттенок которого проходит по кругу:

$$\boldsymbol{c} = 0{,}65\,((1-k)\,\boldsymbol{c}_1 + k\,\boldsymbol{c}_2) + 0{,}35\,\boldsymbol{c}_3, \quad k = 1/2 + 1/2\sin 0{,}9t, \qquad \boldsymbol{h} = 0{,}55 + 0{,}45\cos(0{,}8t + (0;\ 2\pi/3\,;\ 4\pi/3)),$$

и в качестве альбедо берётся $\boldsymbol{c} \odot \boldsymbol{h}$. Три канала $\boldsymbol{h}$ колеблются со сдвигом по фазе, поэтому общий оттенок меняется красный → зелёный → синий → красный, а не просто мигает.

Включение режима и время передаются в **push-константах**: структура `{ float time; uint flags; }` занимает 8 байт и видна только фрагментному шейдеру. Перед каждым рисованием вызывается `vkCmdPushConstants`, бит `flags` включает нетривиальное сэмплирование для этой модели. Благодаря этому структура `ModelUniforms` не получила ни одного поля для управления режимом, а динамический uniform buffer не зависит от того, включён он или нет. Время идёт из функции `update()`.

Блик и свечение

Для третьего задания у материала есть две дополнительные текстуры, прочитанные по тем же координатам (u, v). Цвет блика умножается на тексель карты блика S, а тексель карты свечения E прибавляется к цвету, уже посчитанному по Блинну-Фонгу:

$$C = k_a A + \sum_i E_i \left(k_d \max(\langle \boldsymbol{N}, \boldsymbol{L}_i \rangle, 0) + k_s \odot S \cdot \max(\langle \boldsymbol{N}, \boldsymbol{H}_i \rangle, 0)^p\right) + \gamma\, E,$$

где $k_d = k_a$ — альбедо (текстура, умноженная на оттенок), k_s — цвет блика из интерфейса, γ — множитель свечения (по умолчанию 1). Карта блика говорит, какая часть геометрии блестит: в чёрных текселях множитель $S = 0$ и блик пропадает. Свечение E не зависит ни от источников, ни от нормали и минует расчёт по Блинну-Фонгу, поэтому светится и в полной темноте.

Ответ на вопрос из задания: если у материала нет карты блика или свечения, вместо неё берётся чёрная текстура из одного пикселя. Для блика это умножение на ноль, для свечения — прибавление нуля, поэтому у такого материала нет ни блеска, ни свечения, и в шейдере не нужны ветвления «есть карта или нет» и отдельные конвейеры. Так устроен и встроенный материал «plain».

Карты блика у кирпича, лавы, мрамора, плитки и дерева построены из масок швов и шума: у кирпича блестят только кирпичи, а не раствор, у лавы блестит камень, но не трещины. Карта свечения есть у лавы.

Используемые объекты Vulkan

- **VkBuffer и VmaAllocation**: вершинные и индексные буферы четырёх мешей в памяти устройства; динамический uniform buffer моделей, uniform buffer сцены и два storage buffers источников в памяти процессора; временный staging buffer при каждом копировании.
- **VkImage, VkImageView, VkSampler**: 12 текстур 512 × 512 с полной мип-цепочкой (шесть альбедо, пять карт блика, одна карта свечения), две служебные текстуры 1 × 1, один общий сэмплер.
- **VkCommandPool и VkCommandBuffer**: пул с флагом `TRANSIENT` для одноразовых копирований при запуске.
- **VkDescriptorSetLayout**: три раскладки (сцена, модели с динамическим буфером, материал с тремя парами «изображение–сэмплер»).
- **VkPipelineLayout**: раскладки трёх наборов и диапазон push-констант (8 байт, фрагментный шейдер).
- **VkDescriptorPool и VkDescriptorSet**: набор сцены, набор моделей и по набору на каждый из семи материалов.
- **VkPipeline**: те же три конвейера, что и во второй работе; у вершины появился атрибут с текстурными координатами.

Результаты

На рисунке 1 показана базовая часть: у всех трёх додекаэдров и у пола выбрана одна и та же кирпичная текстура. На гранях виден пятиугольный фрагмент кирпичной кладки, на полу кладка повторяется без швов, а вдали, где кирпичи мельче пикселя, изображение не рябит благодаря мип-уровням.

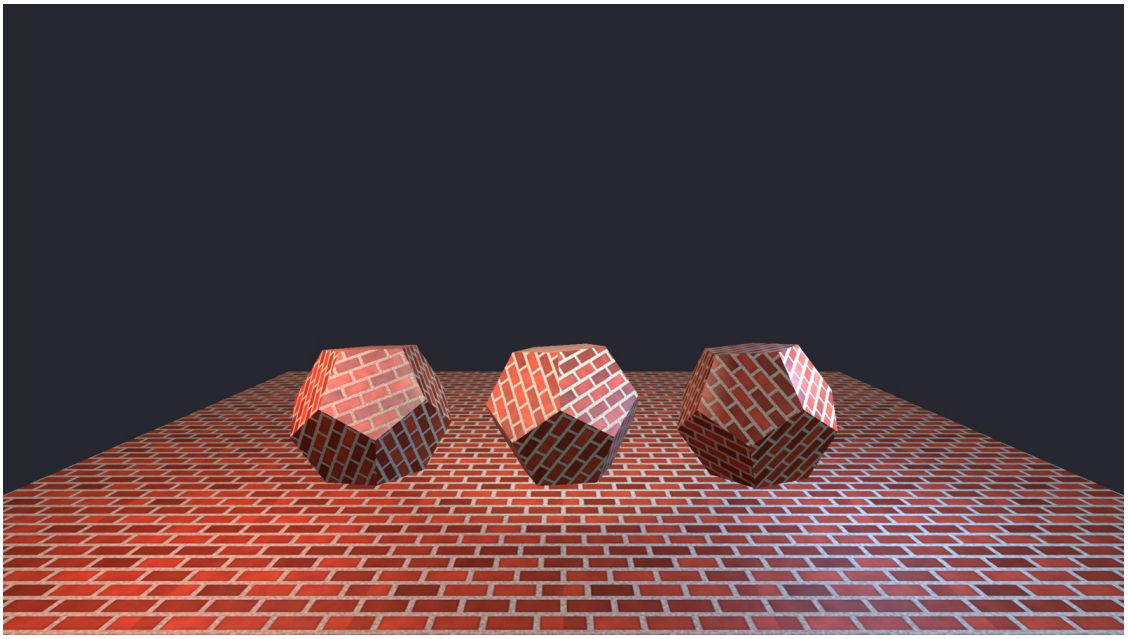


Рисунок 1 — Одна текстура на додекаэдрах и на полу

На рисунке 2 у каждого объекта своя текстура: лава, мрамор и дерево, а пол покрыт плиткой. Текстуры выбираются в интерфейсе независимо. У лавы трещины светятся и при обычном освещении, у мрамора грань, обращённая к направленному источнику, почти белая, а у дерева видны прожилки и швы между досками.

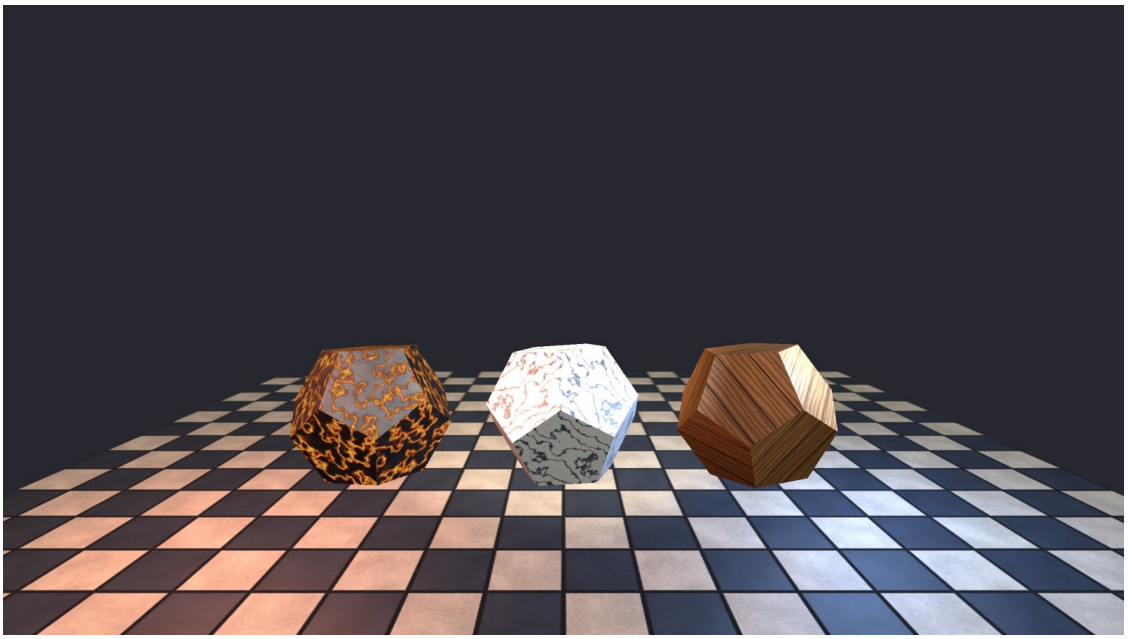


Рисунок 2 — Разные текстуры у объектов и пола

На рисунке 3 вместо картинки взята тестовая сетка: цвет показывает (u, v) (красный растёт вдоль u, зелёный вдоль v), а буква «F» не симметрична, поэтому по ней видно отражение. На полу, который

отображён один раз ((0,0) слева сверху), буква читается нормально, не перевёрнута и не отражена. На гранях додекаэдров буква тоже не отражена, но у соседних граней повёрнута по-разному, потому что у каждой грани свой базис $(\boldsymbol{u}, \boldsymbol{w})$.

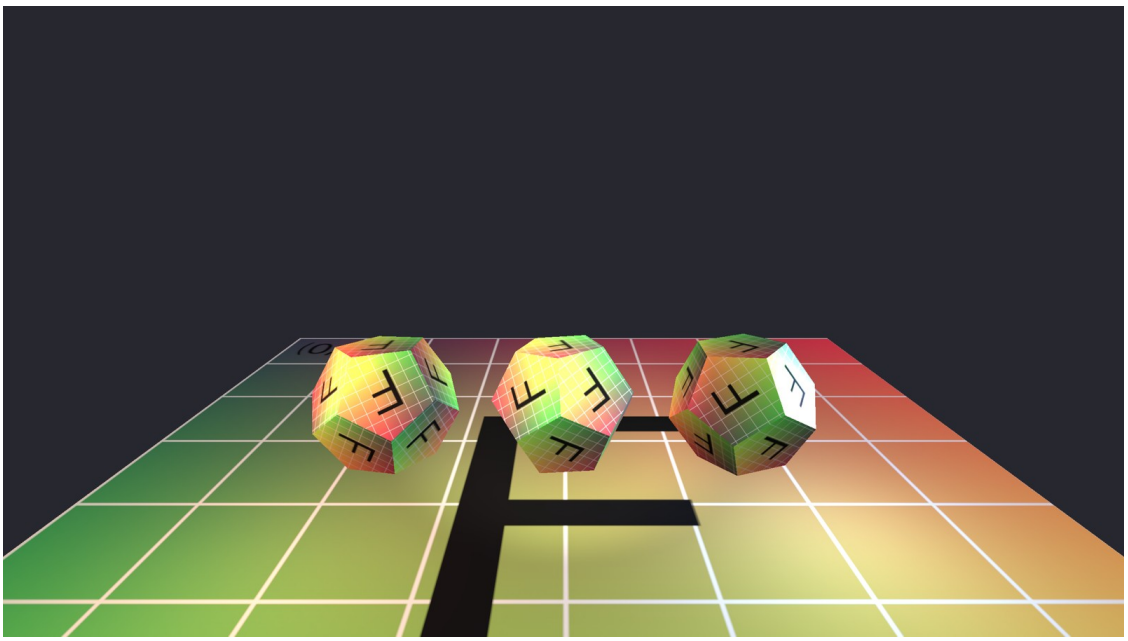


Рисунок 3 — Тестовая сетка: ориентация текстуры на гранях и на полу

На рисунке 4 мраморный объект с включённым нетривиальным сэмплированием в моменты $t = 0$, 3 и 6 секунд. Цвет объекта проходит по оттенкам (розовый, синий, жёлто-зелёный), а прожилки мрамора смещаются, вращаются и «дышат» при изменении масштаба, при этом рисунок складывается из трёх образцов одной и той же текстуры.

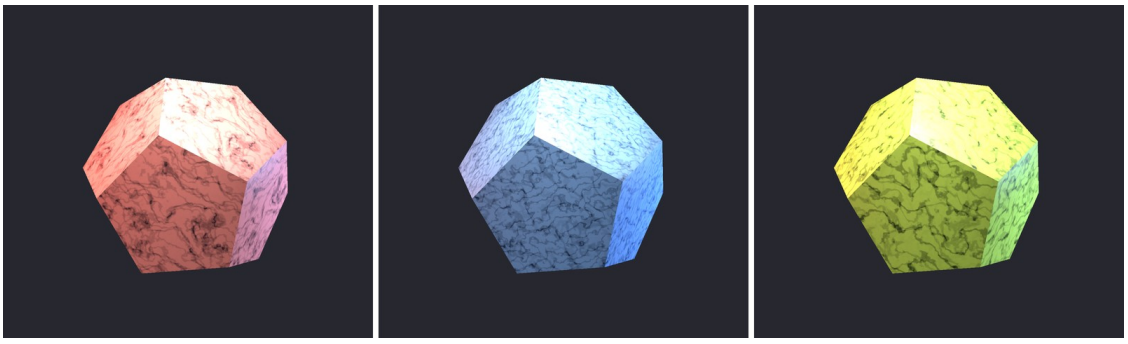


Рисунок 4 — Анимированное сэмплирование в моменты 0, 3 и 6 секунд

На рисунке 5 пол из кирпича освещён точечным источником, направленный свет выключен, рассеянный слабый. Слева у материала есть карта блика: блик лежит на кирпичах и обрывается на швах, раствор остаётся серым. Справа файл карты блика временно убран, и материал получил чёрную текстуру 1 × 1: блик исчез, осталось только рассеянное

отражение. Это та же сцена, поэтому разница получилась именно из-за карты.

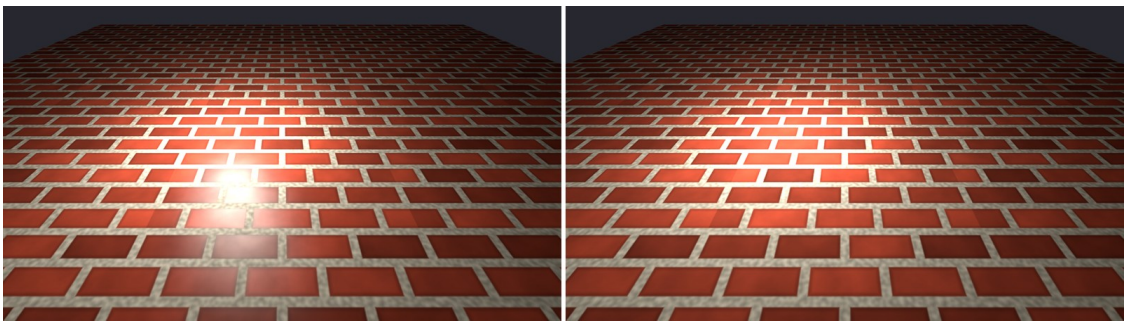


Рисунок 5 — Блик с картой блика (слева) и без неё (справа)

На рисунке 6 направленный свет выключен, рассеянный очень слабый, на сцене лишь один слабый точечный источник. Слева трещины лавы светятся: карта свечения прибавляется после расчёта освещения. Кирпичный и мраморный объекты рядом видны слабо, потому что у них свечения нет. Справа та же сцена, у лавы убран файл карты свечения: она получила чёрную текстуру и стала просто тёмным камнем.

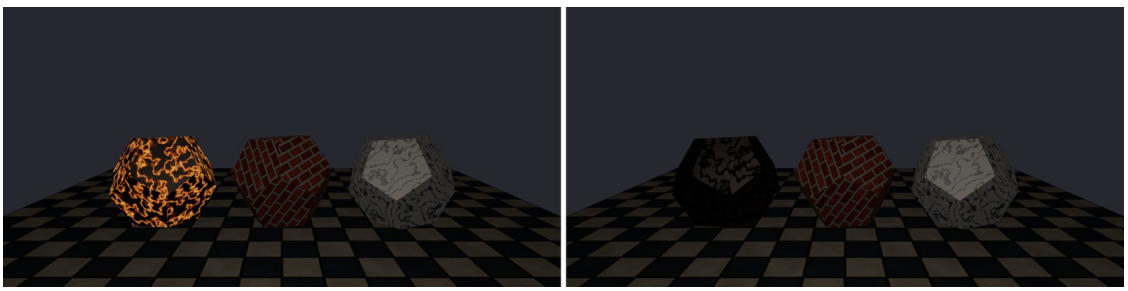


Рисунок 6 — Свечение в темноте с картой свечения (слева) и без неё (справа)

На рисунке 7 показан интерфейс: выпадающий список материалов выбранного объекта (рядом с номером объекта написано название текстуры), сведения о картах блика и свечения у выбранного материала, оттенок, цвет и коэффициент блика, множитель свечения и флажок нетривиального сэмплирования.

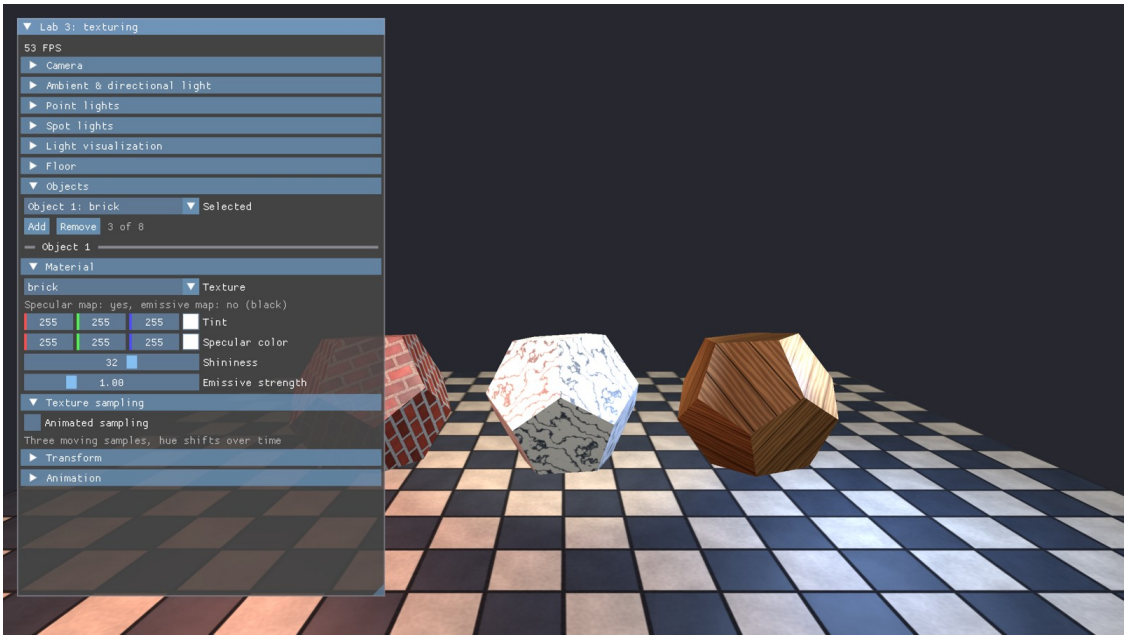


Рисунок 7 — Выбор текстуры и параметры материала в интерфейсе

Автоматические тесты (713 проверок) проходят, а слой валидации Vulkan не выдаёт сообщений ни во время работы, ни при завершении программы, ни на одном из кадров для рисунков. Тесты проверяют текстурные координаты (круг радиуса 0,5, равные стороны, отсутствие отражения), число уровней мип-цепочки, загрузку изображений, поиск материалов по именам файлов, назначение материалов новым объектам и то, что нетривиальное сэмплирование включено ровно у одного объекта начальной сцены. Остальные проверки остались от прошлых работ.

Выводы

Я научилась загружать изображения и превращать их в текстуры в памяти GPU: писать в staging buffer, копировать в `VkImage` и строить мип-цепочку блитами, меняя layout каждого уровня барьерами. На практике поняла, зачем альбедо хранится в sRGB-формате, а карта блика нет: sRGB переводит цвет в линейные значения, а данные должны остаться как есть. Текстурные координаты додекаэдра я вывела из базиса грани и разобралась, откуда берётся знак минус у v: в изображении строки идут сверху вниз, и без этого текстура на каждой грани была бы отражена.

Один динамический uniform buffer вместо 57 отдельных показал, что смещение к данным модели можно передавать прямо в команде привязки набора дескрипторов, но шаг между элементами зависит от устройства (`minUniformBufferOffsetAlignment`). Разделение наборов на сцену,

модели и материал помогло переключать текстуры одной командой, а push-константы оказались удобным способом управлять режимом шейдера для отдельной модели, не меняя данные в буфере. Чёрная текстура из одного пикселя вместо отсутствующих карт блика и свечения избавила от ветвлений в шейдере и от отдельных конвейеров: блик умножается на ноль, а свечение прибавляет ноль.